



Department of Data Science

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date: 03-10-2026 | Venue: 1st Floor, A & B Blocks

Field	Details
Project Title	Student Record Lookup Using Binary Search
Team Name / Team No.	S4/17
Year / Semester / Section	II Year B.Tech / I Sem / _C_
Algorithm Paradigm Used	Divide & Conquer (Binary Search)
Programming Language / Tools	Python / Python IDLE
Faculty Mentor	SANDHYA MAM

Team Members

S.No	Name of the Student	Roll No.	Role
1	M.SAI RAKESH	25MVCSDR0227	Team Lead
2	CH.SRI CHARAN	25MVCSDR0198	Data Collection & Analysis
3	C.SAMANTH REDDY	25MVCSDR0199	Digital Twin & Simulation
4	J.SAI CHARAN	25MVCSDR0215	Smart Cooling & Dashboard

1. Problem Statement

The project addresses efficient retrieval of student records using unique roll numbers. A collection of student records is stored and sorted by roll number. Given one or more roll numbers, the system searches the records and displays the complete matching student details. The project also counts the number of key comparisons and compares Binary Search with a naive Linear Search approach.

Inputs: A list of student records containing Roll Number, Name, Department and Marks; one or more roll numbers to search.

Outputs: Matching student details when the roll number exists; a not-found message otherwise; number of comparisons for each search and comparison with Linear Search.

Constraints: Roll numbers are unique; records must be sorted by roll number before Binary Search; the search key is a roll number; the algorithm should avoid unnecessary sequential checking.

2. Objectives

1. Retrieve student details efficiently using Binary Search on sorted roll numbers.
2. Count the number of roll-number comparisons made during each search.
3. Compare Binary Search performance with a naive Linear Search approach.
4. Display clear student information and search results for valid and invalid roll numbers.

3. Proposed Approach

The project uses the Divide & Conquer paradigm through Binary Search. After sorting the records by roll number, Binary Search checks the middle record and eliminates half of the remaining search range after each comparison. This makes it suitable for repeated lookup of unique roll numbers in a sorted dataset. Linear Search was considered as the naive alternative because it checks records one by one and provides a direct baseline for comparison.

Data used in the demonstration is sorted by roll number before searching. For every query, the program tracks the number of comparisons made by Binary Search and, for comparison, the number made by Linear Search.

Why this approach: Binary Search reduces the search space by half at every step, giving $O(\log n)$ search time, while Linear Search may inspect every record and takes $O(n)$ time in the worst case.

4. Algorithm / Pseudocode

Algorithm:

1. Read the student records and store each record as (Roll No., Name, Department, Marks).
2. Sort the records in ascending order of Roll Number.
3. Read a roll number to be searched and set $low = 0$ and $high = n - 1$.
4. While $low \leq high$, calculate $mid = (low + high) // 2$ and increase the comparison counter.
5. If the middle record's roll number equals the search key, display the complete student details and stop.
6. If the search key is smaller, move $high$ to $mid - 1$; otherwise move low to $mid + 1$. If the range becomes empty, display 'Record Not Found'. Repeat for additional queries if required.

5. Complexity Analysis

For n student records, Binary Search requires $O(\log n)$ time on average and in the worst case, while a successful search can finish in $O(1)$ when the first middle element matches. The iterative implementation uses $O(1)$ auxiliary space. Sorting the records before searching is a separate preprocessing step; the lookup complexity reported here is for the Binary Search operation. The naive Linear Search baseline requires $O(n)$ time in the worst case.

Complexity	Best Case	Average Case	Worst Case
Time	$O(1)$	$O(\log n)$	$O(\log n)$
Space	$O(1)$ auxiliary	$O(1)$ auxiliary	$O(1)$ auxiliary

6. Implementation

Main modules/functions: record storage, sorting by roll number, Binary Search, Linear Search comparison, comparison counting, and formatted result display.

Data structures: A list of student records, where each record contains Roll Number, Name, Department and Marks.

Binary Search module: maintains low, high and mid indices and compares the search roll number with the middle record.

Comparison module: counts key comparisons for Binary Search and Linear Search so that the two approaches can be compared for the same query.

Platform/Tool: Python, developed and executed using Python IDLE.

7. Results and Test Cases

Sample dataset (sorted by Roll No.): 101-Sai-CSE-91, 103-Arjun-IT-88, 105-Ravi-CSE-82, 108-Kiran-CSE-95, 110-Rahul-ECE-76, 112-Priya-ECE-86, 115-Vikram-MECH-79.

Test Case 1 — Input Roll No.: 108. Binary Search output: Kiran, CSE, 95 marks; comparisons = 1. Linear Search comparisons = 4.

Test Case 2 — Input Roll No.: 115. Binary Search output: Vikram, MECH, 79 marks; comparisons = 3. Linear Search comparisons = 7.

Test Case 3 — Input Roll No.: 104. Output: Record Not Found; Binary Search comparisons = 3. Linear Search comparisons = 7.

Observation: For the same dataset, Binary Search reaches the required record or determines that it is absent with fewer comparisons than sequential Linear Search in these test cases. Screenshots of the program execution can be attached to this section.

8. Conclusion and Future Scope

The project demonstrates that Binary Search can provide efficient student-record lookup when records are sorted by a unique roll number. It displays complete student details, reports comparison counts, and provides a direct performance comparison with Linear Search.

Future scope includes supporting larger datasets, file/database-based student records, multiple-query benchmarking, a graphical user interface, and automatic generation of performance reports and comparison charts.

9. References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python. Wiley, 2013.
- [3] Python Software Foundation, "Python Documentation," Python.org.
- [4] APSH 2026 project implementation and test dataset developed by the project team.

